

A Deep Dive Into the Bus System

Member 1: Go, Justin

Member 2: Leung, Jillianne

Member 3: Luna, Jacoba

Member 4: Montaña Rovin

Member 5: Teoxon, Jat

Topic Theme:

How Bus Arbitration Works – Resolving Contention When Multiple Devices Need the Bus

Main concept:

Modern computer systems contain multiple components that need access to a common communication channel called the system bus. The CPU, memory controller, DMA controller, storage devices, and peripherals may all attempt to use the bus simultaneously. Without a mechanism to control access, data collisions and system instability would occur.

A computer bus is a shared communication pathway. Only one device can transmit data at a time. Bus arbitration is the mechanism that decides which device gets control of the bus when multiple devices request access simultaneously.

SECTION 1: Purpose of a bus arbiter and Foundational Concepts

A computer bus is a communication channel carrying data, addresses, and control signals between different components. The bus is shared between multiple devices, and only one device can drive it at a time. If two devices try to transmit simultaneously, their signals corrupt each other, creating what is known as a 'bus conflict.'

A bus arbiter is a hardware unit that works towards preventing these bus conflicts from happening. Devices signal to the bus arbiter that they would like to use the bus by sending a Bus Request (BR) line, and the arbiter responds with a Bus Grant (BG) line to one device at a time. The device that receives the BG will then assert Bus Busy (BBSY) to signal that the bus is in use. When the device is finished transferring data, it lifts the BBSY. This method not only ensures that bus conflicts won't occur, but also that the bus is used in an orderly manner and that all BG lines are responded to.

Prior to examining specific hardware implementations or the interactive simulator, it is essential to establish that mediating this shared access involves deeper systemic challenges. The arbitration process is governed by three fundamental design considerations:

1.1 The Fairness vs. Urgency Trade-off

System architects must balance equitable access among all devices against the prioritization of critical operations. For example, a CPU executing a power-cycle interrupt requires more immediate access than a storage drive performing a background data transfer. While fairness prevents low-priority devices from experiencing indefinite delays (starvation), prioritizing urgency guarantees that the system remains responsive to time-sensitive operations. Arbitration algorithms are primarily categorized by how they balance these two opposing requirements.

1.2 Centralized vs. Distributed Decision-Making

Arbitration logic is implemented either through a single dedicated hardware unit (centralized) or dispersed across multiple devices on the bus (distributed). Centralized architectures offer simplified control logic but introduce a single point of failure. Conversely, distributed architectures provide greater fault tolerance but necessitate highly complex coordination protocols to function effectively.

1.3 The Hidden Cost: Arbitration Overhead

The arbitration process inherently consumes system clock cycles. Devices must assert request lines, the arbiter must evaluate competing priorities, and grant signals must propagate back to the requesting components. During this evaluation phase, the bus remains idle. This delay is known as arbitration latency. Selecting an inefficient arbitration mode for a specific system workload can consume excessive bus bandwidth, ultimately degrading overall system performance.

Evaluating these three considerations—fairness versus urgency, centralized versus distributed logic, and latency overhead—provides a framework for analyzing bus arbitration systems. The subsequent sections and the interactive simulator will demonstrate how different arbitration modes prioritize specific parameters at the expense of others.

SECTION 2: Types of arbitration modes

2.1 Fixed Priority

The arbiter holds a simple priority encoder. When multiple BR signals are asserted simultaneously, the encoder selects the highest-priority line and asserts BG only for that device. Lower-priority devices receive no grant and must wait. If the high-priority device keeps requesting, lower devices may wait indefinitely, a condition called starvation. Fixed priority is ideal when device urgency is genuinely asymmetric. A CPU handling a real-time interrupt is more urgent than a disk prefetch. But in a system where devices have similar urgency, fixed priority is unfair and can cause significant performance degradation for low-priority devices.

- Pros

- Deterministic, simple hardware (priority encoder)
 - Critical devices always get immediate access
- Cons
 - Low-priority devices (Disk I/O) may never get the bus if high-priority devices keep requesting

2.2 Round-Robin

The arbiter maintains a rotating pointer (also called a rotating priority register). After granting the bus to device N, the pointer advances to N+1, making N+1 the highest priority for the next cycle. If N+1 has no request, the arbiter checks N+2, and so on in a circle until it finds a requester. This guarantees that every device will wait at most N-1 cycles before getting access, where N is the total number of requesters. It is the standard mode for multimedia buses where latency fairness matters, for instance, a system in which audio DMA must not be blocked indefinitely by a CPU doing a file transfer.

- Pros
 - No starvation
 - Fair and predictable access
- Cons
 - More complex hardware
 - High priority devices may wait longer than fixed priority

2.3 Daisy Chain

Daisy-chain arbitration uses a single bus grant (BG) wire that threads through all devices in sequence. The central arbiter asserts BG when the bus is free. If the first device in the chain has a pending request, it intercepts the signal, claims the bus, and sends nothing downstream. If it has no request, it immediately passes BG to the next device, and so on. The advantage of this is hardware as it only needs one grant wire and no individual grant lines from the arbiter to each device. The disadvantage is that priority is determined entirely by physical position in the chain. The first device is always the highest priority, and the last can starve. Additionally, the grant propagation delay through the chain adds latency proportional to chain length.

- Pros:
 - Very few wires (one GNT line shared across chain)
 - Simple and low cost for small numbers of devices
- Cons:
 - Priority is fixed by physical position
 - Speed degrades with chain length (propagation delay)
 - Single broken device can break the chain

SECTION 3: Bus Arbiter Simulator

Users see a diagram with:

- Three devices: CPU (left), DMA Controller (center), Disk I/O (right)

- One shared bus (horizontal line connecting all three)
- Arbiter block (center-top, acting as traffic cop)

Users can:

1. Click “Request Bus” buttons on any device at any time
2. Choose an arbitration mode:
 - Fixed Priority (CPU > DMA > Disk I/O)
 - Round-Robin (cyclic rotation)
 - Daisy Chain (priority passes down the line)
3. Watch the simulation decide who gets the bus
4. See visual feedback:
 - Requesting device blinks yellow
 - Granted device turns green and a pulse travels along the bus
 - Denied devices turn red and wait in queue
5. View a transaction log showing:
 - “Cycle 4: CPU requested bus”
 - “Cycle 5: DMA requested bus”
 - “Cycle 6: Arbiter granted bus to CPU (fixed priority)”
 - “Cycle 10: CPU released bus”

User interaction flow:

1. User selects arbitration mode (dropdown)
2. User clicks “Request Bus” on CPU → CPU blinks yellow, added to queue
3. User clicks “Request Bus” on DMA → DMA blinks yellow, added to queue
4. User clicks “Start Arbitration” button (or auto-runs every cycle)
5. Arbiter processes queue based on priority mode
6. Winner turns green, pulse animation travels bus
7. Losers stay red, remain in queue
8. After 2 seconds, winner releases bus → turns gray
9. Next cycle begins, next device in queue gets granted

SECTION 4: Tech Stack and Style Guide

Our team has chosen the following components and frameworks for our tech stack while ensuring that they are fully compliant with the CSARCH2 project specifications and compatible as well.

1. Runtime Environment: [Node.js](#) 26
 - This will be the underlying JavaScript runtime environment for building and running the project as specified in the project specifications.

2. Main Framework: Astro 6
 - Astro 6 will be our primary static site generator for the virtual exhibit page we will make.
3. Content Authoring: Markdown Extended (MDX)
 - All textual content, such as historical contexts and deep-dive explanations about bus arbitration will be written in .mdx files. MDX will allow us to embed our interactive React components directly within the Markdown text.
4. Interactive Elements: [React.js](#) (.jsx)
 - React will be used to build the simulator component that will visualize the bus arbitration process. React's useState and useEffect hooks will be suited for handling the tracking of request queues of CPU, DMA, and Disk I/O, managing the active arbitration mode, and updating the transaction log without reloading the page.
5. Styling: Tailwind CSS
 - This framework will be used to handle all layout, styling, and mobile-responsive behaviors that our virtual exhibit will require. We chose Tailwind since it has features that allow it to integrate into Astro without hiccups.
6. Animation: Motion (Formerly Framer Motion)
 - Motion will be used for specific animations that will portray data as pulses traveling through the buses, mentioned in our interaction flow. It can help with smooth, hardware-accelerated animations that are easier to program than raw CSS keyframes.
7. Text Layout Engine: [Pretext.js](#)
 - We chose [Pretext.js](#) for layouting and positioning dynamic, real-time explanations and educational overlays while the simulation is running. As the simulator runs, we will display contextual explanations detailing exactly *why* a device is granted or denied access based on the current arbitration mode. Because these explanation blocks will constantly shift in size and length, Pretext.js calculates their text heights and line breaks using pure arithmetic instead of querying the DOM.

Bus Arbitration System



Mode **Fixed Priority** ▾

CPU > DMA > DISK I/O

Cycle 8

BUS ARBITER

System bus data / address / control

CPU

granted

Request Bus

DMA Controller

requesting

Request Bus

DISK I/O

denied

Request Bus

Transaction log

cycle 4 · CPU requested bus

cycle 5 · DMA requested bus

cycle 6 · arbiter granted bus → CPU (fixed priority)

cycle 6 · Disk I/O denied → queued

style

Palette · state-driven

Typography

Bus

Accp

Req

Deny

Idle

IBM Plex Sans

Body text